



CODTECH

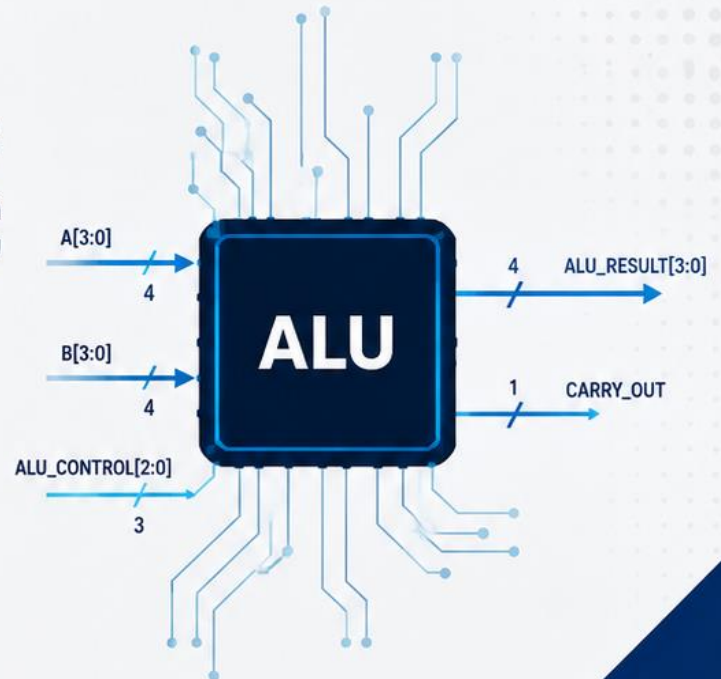
— IT SOLUTIONS —

INNOVATE • INTEGRATE • INSPIRE

PROJECT REPORT

DESIGN AND VERIFICATION OF A 4-BIT ARITHMETIC LOGIC UNIT (ALU) USING VERILOG HDL

A VLSI DESIGN AND SIMULATION PROJECT



SUBMITTED BY
AYUSH SIKARWAR
VLSI INTERN



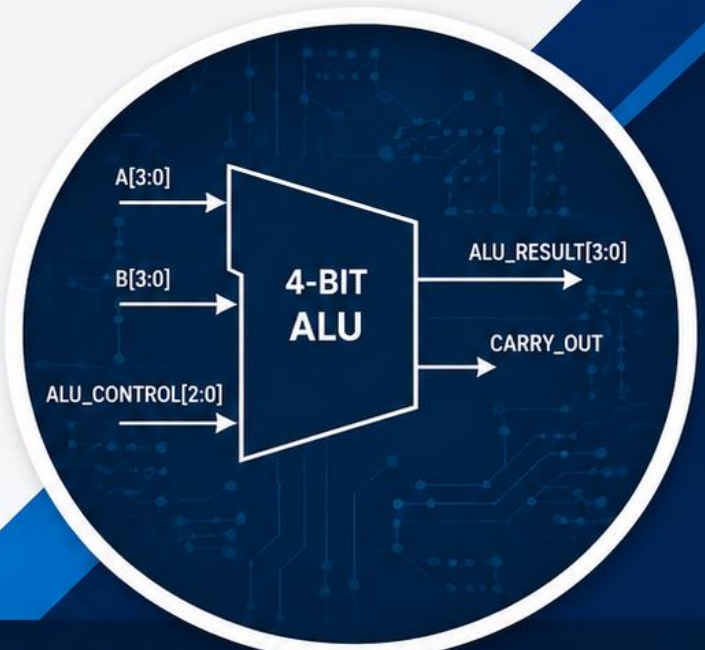
ORGANIZATION
CODTECH IT SOLUTIONS



DOMAIN
VLSI DESIGN



DEPARTMENT
**ELECTRONICS AND
COMMUNICATION ENGINEERING**



INNOVATIVE
SOLUTIONS



TECHNOLOGY
DRIVEN



PRECISE
RESULTS



BUILDING THE
FUTURE TOGETHER

OVERVIEW

- Developed and programmed a 4-bit Arithmetic Logic Unit (ALU) in Verilog HDL.
- Designed combinational digital circuit to perform several arithmetic and logical operations.
- Performed operations like Addition, Subtraction, AND, OR, XOR, NOT, Increment, and Decrement (modify as per your implementation).
- Improved readability, reusability and scalability of code by using modular Verilog programming.
- Tested and tested the design using a Verilog simulation tool to verify correct working.
- Showed how the ALU, a basic element of contemporary processors and digital systems works.
- Acquired hands-on experience in functional verification, digital logic design, HDL programming and hardware modeling.
- Enhanced conceptual knowledge of combinational circuits, binary arithmetic and VLSI design concepts.
- Developed a scalable structure which can be extended to 8-bit, 16-bit or 32-bit ALU architectures for advanced processor application.

OBJECTIVE

This project aims to design, implement and verify a 4-bit Arithmetic Logic Unit (ALU) in Verilog HDL. The objective of the project is to design a digital circuit that is capable of carrying out basic arithmetic and logical operations and also to give hands-on experience to the students with hardware description language (HDL) programming, RTL design and combinational logic. It also includes simulation validation of the ALU functionality, which enhances the knowledge and understanding of digital system design and VLSI design principles. Moreover, the project is the basis for the design of more complex and scalable processor components that can be employed in today's digital and embedded systems.

INTRODUCTION

One of the most crucial parts of a computer processor and digital systems is an Arithmetic Logic Unit (ALU). It's used to perform arithmetic functions like adding and subtracting, and logical functions like AND, OR, XOR, NOT. The ALU of a processor plays a key role in its performance, as it determines the efficiency and reliability of the processor.

In this project, a 4-bit ALU is designed and implemented in the Verilog Hardware Description Language (HDL), which is one of the most popular Hardware Description languages (HDLs) used in digital circuit design and VLSI development. The design is done modularly, meaning that the desired operation is determined by control signals. The ALU has been checked for its functionality using simulation to ensure the correct working for various input conditions.

This project gives practical experience to the students in Digital system design, RTL coding, functional simulation and hardware verification. It also provides a solid basis to grasp the architecture of the processors and the advanced concepts in VLSI design, which is a stepping stone in the design of complex digital systems.

BLOCK DIAGRAM DESCRIPTION

The 4-bit ALU (alu_4bit) takes two 4-bit data inputs, one 3-bit control input, and generates a 4-bit data output along with two status flags.

Input Ports:

- 1.a [3:0]: 4-bit input operand A
- 2.b [3:0]: 4-bit input operand B
- 3.alu_control [2:0]: 3-bit selection lines to decide the ALU operation.

Output ports:

- 1.alu_result [3:0]: 4-bit output displaying the operation result
- 2.carry_out: 1-bit flag indicating an arithmetic carry or borrow
- 3.zero_flag: 1-bit flag indicating if the output result is zero

Functional Table (Operation Codes)

Control Code	Operation Type		Operation Description
3'b000	Arithmetic Addition		$\text{alu_result} = a + b$
3'b001	Arithmetic Subtraction		$\text{alu_result} = a - b$
3'b010	Logical	Bitwise AND	$\text{alu_result} = a \& b$
3'b011	Logical	Bitwise OR	$\text{alu_result} = a b$
3'b100	Logical	Bitwise XOR	$\text{alu_result} = a \wedge b$
3'b101	Logical	Bitwise NOR	$\text{alu_result} = \sim(a b)$
3'b110	Logical	Bitwise NAND	$\text{alu_result} = \sim(a \& b)$
3'b111	Logical	Bitwise XNOR	$\text{alu_result} = \sim(a \wedge b)$

RTL ARCHITECTURE AND CODE WALKTHROUGH

The Design implementation ensures robust hardware generation through the following practices:

Latch Prevention:

alu_result and carry_out are initialized to default values at the beginning of the always @(*) block to avoid unwanted latches

Combinational Logic:

The module uses a standard case statement sensitive to all input transitions.

Flag generation:

1.Carry Flag:

Calculated directly during addition. For subtraction, it indicates a borrow ($a < b$).

2.Zero Flag:

A separate conditional block checks if alu_result equals 4'b0000 and dynamically updates the zero_flag

TESTBENCH AND VERIFICATION STRATEGY

The testbench (tb_alu_4bit) creates a self-contained simulation environment to validate all operations

Verification plan Waveform Generation:

\$dumpfile and \$dumpvars are used to generate VCD (Value Change Dump) logs for waveform viewing.

Console Monitoring:

\$monitor logs data in real-time whenever inputs or outputs change.

Stimulus Driving: Comprehensive test cases are injected with a #10 time unit delay to observe stable responses.

Executed Test Cases:

1.Addition:

$4 + 5 = 9$ (No Carry). Addition with Carry: $12 + 6 = 18$ (Result = 2, Carry = 1).

2.Subtraction:

$10 - 4 = 6$ (No Borrow).

3.Bitwise AND:

Checks masking behavior.

4.Bitwise OR:

Checks logical addition.

5.Bitwise XOR:

Identical inputs generate all zeros, validating the zero_flag.

6.Shift Left: Shifts operand A by one bit positions

SOURCE CODE (alu_4bit.v)-

```
``verilog
`timescale 1ns / 1ps

module alu_4bit (
    input wire [3:0] a,          // 4-bit input operand A
    input wire [3:0] b,          // 4-bit input operand B
    input wire [2:0] alu_control, // 3-bit control signal to select operation
    output reg [3:0] alu_result, // 4-bit output result of the operation
    output reg      carry_out,   // Carry out flag for addition
    output reg      zero_flag    // Zero flag set high if alu_result is 0
);
// Combinational ALU Logic
always @(*) begin

    // Default outputs
    alu_result = 4'b0000;
    carry_out  = 1'b0;

    case (alu_control)

        // 000 : Addition
        3'b000:
```

$\{\text{carry_out}, \text{alu_result}\} = a + b;$

// 001 : Subtraction

3'b001: begin

alu_result = a - b;

carry_out = (a < b); // Borrow Indicator

end

// 010 : Bitwise AND

3'b010:

alu_result = a & b;

// 011 : Bitwise OR

3'b011:

alu_result = a | b;

// 100 : Bitwise XOR

3'b100:

alu_result = a ^ b;

// 101 : Bitwise NOR

3'b101:

alu_result = ~(a | b);

// 110 : Logical Left Shift

3'b110:

```
alu_result = a << 1;
```

```
// 111 : Logical Right Shift
```

```
3'b111:
```

```
alu_result = a >> 1;
```

```
// Default Case
```

```
default: begin
```

```
alu_result = 4'b0000;
```

```
carry_out = 1'b0;
```

```
end
```

```
endcase
```

```
// Zero Flag Logic
```

```
zero_flag = (alu_result == 4'b0000);
```

```
end
```

```
endmodule
```

```
```
```

```

```

# TEST BENCH CODE

```
//
```

```
=====
```

```
=====
```

```
// COMPANY NAME: CODTECH IT SOLUTIONS
```

```
// INTERN NAME: AYUSH SIKARWAR
```

```
// INTERN ID: CITS5568
```

```
// DOMAIN: VLSI
```

```
// TASK TITLE: Design and Implementation of a Basic 4-bit ALU
```

```
//
```

```
=====
```

```
=====
```

```
`timescale 1ns / 1ps
```

```
module tb_alu_4bit;
```

```
 // Inputs to the Device Under Test (DUT)
```

```
 reg [3:0] a;
```

```
 reg [3:0] b;
```

```
 reg [2:0] alu_control;
```

```
 // Outputs from the Device Under Test (DUT)
```

```
wire [3:0] alu_result;
```

```
wire carry_out;
```

```
wire zero_flag;
```

```
// Instantiate the 4-bit ALU module
```

```
alu_4bit uut (
```

```
 .a(a),
```

```
 .b(b),
```

```
 .alu_control(alu_control),
```

```
 .alu_result(alu_result),
```

```
 .carry_out(carry_out),
```

```
 .zero_flag(zero_flag)
```

```
);
```

```
// Stimulus block
```

```
initial begin $dumpfile("dump.vcd"); $dumpvars(0, tb_alu_4bit);
a=4'b0000; b=4'b0000; alu_control=3'b000;
```

```
 // Initialize inputs
```

```
 a = 4'b0000;
```

```
 b = 4'b0000;
```

```
 alu_control = 3'b000;
```

```
// Monitor window for console logging
```

```
$monitor("Time=%0dns | Control=%b | A=%b B=%b | Result=%b
| Carry=%b | Zero=%b",
 $time, alu_control, a, b, alu_result, carry_out, zero_flag);

#10;

// Test Case 1: Addition ($4 + 5 = 9$)
a = 4'b0100; b = 4'b0101; alu_control = 3'b000; #10;

// Test Case 2: Addition with Carry ($12 + 6 = 18 \rightarrow$ Result 2, Carry
1)
a = 4'b1100; b = 4'b0110; alu_control = 3'b000; #10;

// Test Case 3: Subtraction ($10 - 4 = 6$)
a = 4'b1010; b = 4'b0100; alu_control = 3'b001; #10;

// Test Case 4: Bitwise AND
a = 4'b1100; b = 4'b1010; alu_control = 3'b010; #10;

// Test Case 5: Bitwise OR
a = 4'b1100; b = 4'b1010; alu_control = 3'b011; #10;

// Test Case 6: Bitwise XOR (Should trigger zero flag if identical)
a = 4'b1111; b = 4'b1111; alu_control = 3'b100; #10;
```

```
// Test Case 7: Left Shift A
```

```
a = 4'b0011; alu_control = 3'b110; #10;
```

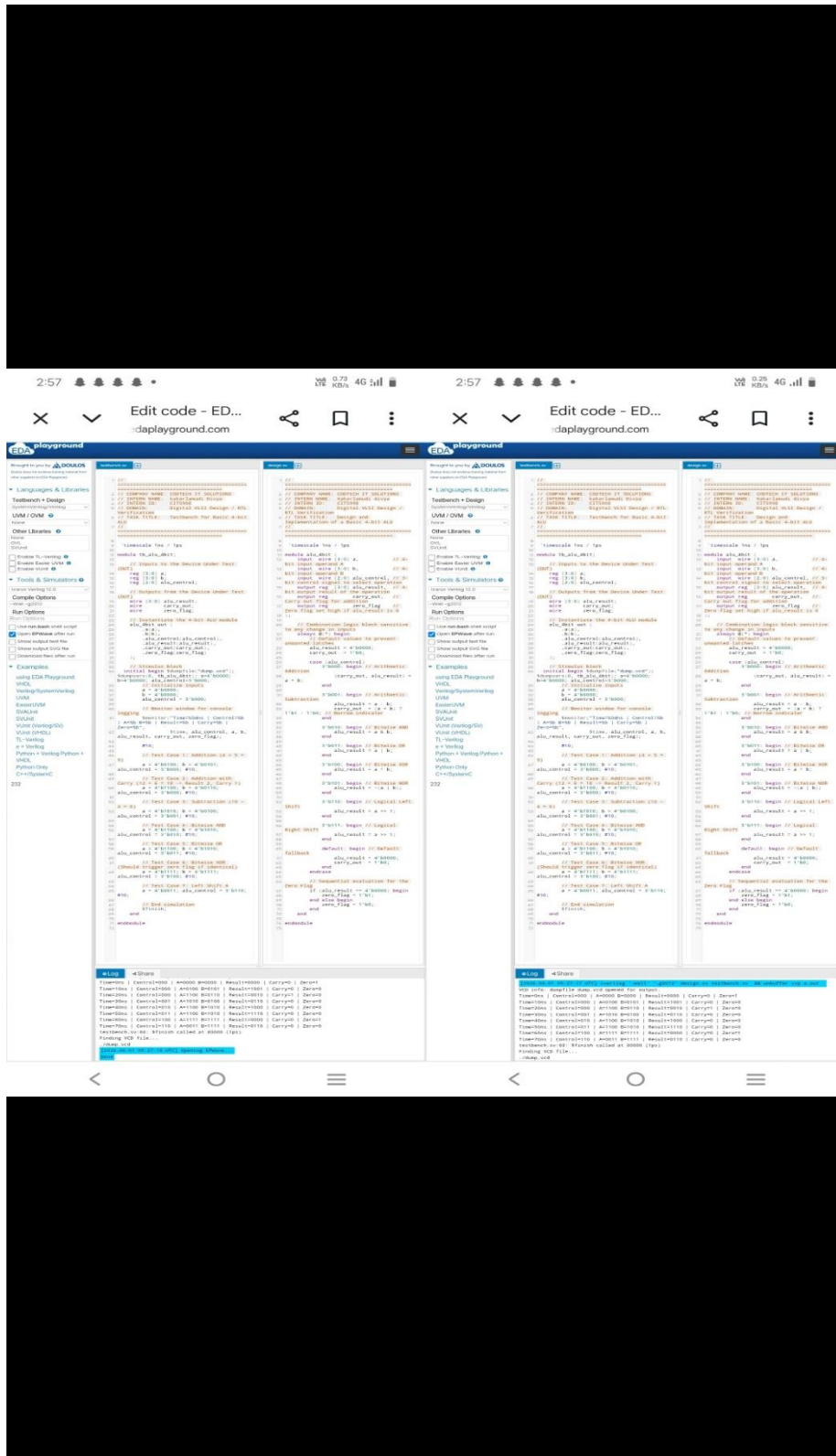
```
// End simulation
```

```
$finish;
```

```
end
```

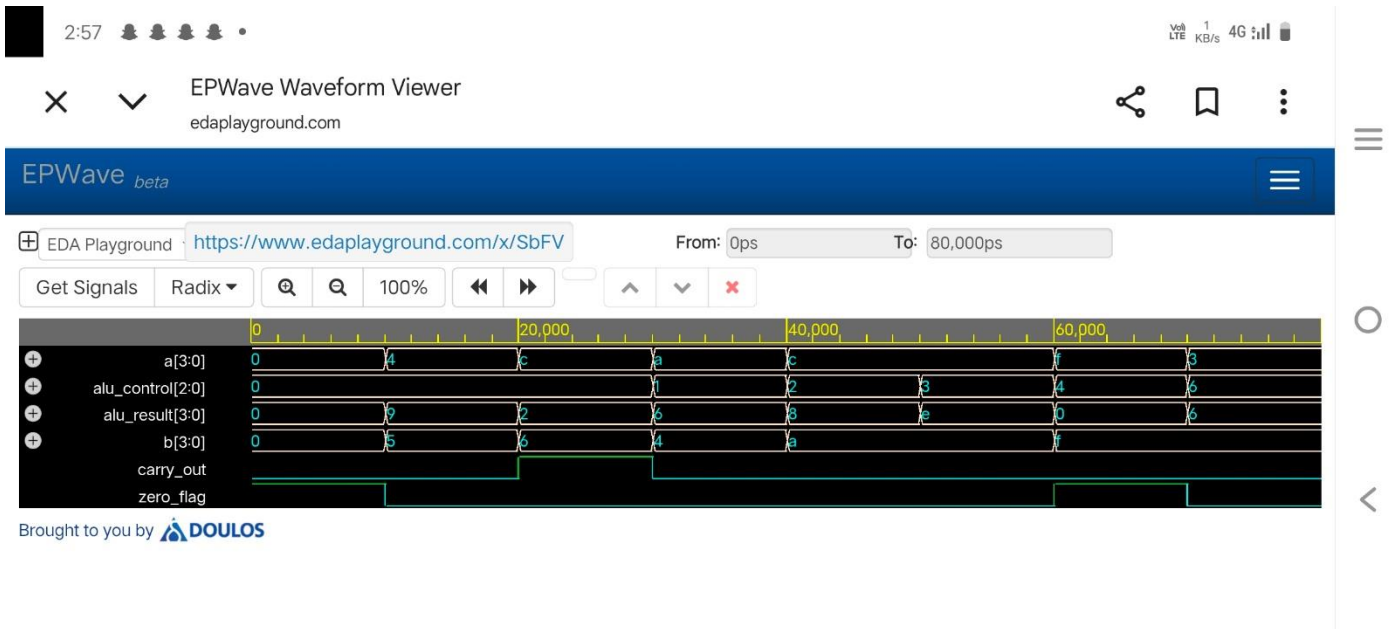
```
endmodule
```

# SIMULATION RESULTS & OUTPUTS





## TIMING WAVEFORM (epwave window)



# CONCLUSION

The 4-bit ALU design was successfully implemented and verified using Verilog HDL. The simulation logs and waveforms confirm that both the arithmetic-logic hardware and the conditional status flags (zero\_flag, carry\_out) function exactly as specified in the architecture requirements.

# CHALLENGES FACED

- Knowledge of basic and syntax of Verilog HDL.
- Creating the ALU to perform several arithmetic and logical operations by using a 3-bit control signal.
- Correctly carry and borrow for numbers in arithmetic operations.
- Troubleshooting coding mistakes and solving simulation problems.
- Testing each operation for various combinations of inputs, and verifying the results.
- Maintaining the modularity and efficiency of the design without introducing any logic conflicts.

# FUTURE IMPROVEMENTS

- Expand the ALU to 4-bit, 8-bit, 16-bit, 32-bit or 64-bit.
- Add more complex operations including multiplication, division, shift, rotate, and comparison functions.
- Use processor status flags: Zero, Carry, Overflow, Negative and Parity.
- Use of high speed adder architectures such as Carry Look-Ahead Adders (CLA) to optimize the design.
- Implement and test design on FPGA development boards.
- Construct a simple processor by incorporating the ALU, registers, and memory.
- Synthesize, Timing and Optimize power consumption using industry standard VLSI tools.

# LEARNING OUTCOMES

- Gained an understanding of basics of Verilog HDL/RTL design.
- Developed hands on experience designing and implementing 4 bit ALU.
- Enhancement of knowledge about combinational logic and digital system design.
- Learned simulation, debugging and functional verification skills.
- Improved understanding of arithmetic and logical operations of processors.
- Understood idea of Modular Coding in Hardware Design.
- Established good background in FPGA, ASIC and VLSI design projects.

# APPLICATIONS OF THE PROJECT

- Computer processors (CPU) for performing arithmetic and logical operations.
- Microcontrollers and embedded systems.
- Digital Hardware Development with FPGAs.
- Here are some of the most popular courses in ASIC and VLSI chip design.
- Digital Signal Processing (DSP) systems.
- Robotics and industrial automation system.
- Communication and networking equipment.
- Consumer electronics, including calculators, smart devices and IoT systems.
- Digital electronics and computer architecture education labs.
- Investigation and development of semiconductor and digital system design.

# REFERENCES

1. M. Morris Mano and Michael D. Ciletti, Digital Design, 6th Edition, Pearson Education, 2018.
2. Samir Palnitkar, Verilog HDL: A Guide to Digital Design and Synthesis, 2nd Edition, Pearson Education, 2003.
3. Stephen Brown, Zvonko Vranesic, Fundamentals of Digital Logic with Verilog Design, 3rd Edition, McGraw-Hill Education, 2013.
4. Institute of Electrical and Electronics Engineers (IEEE) IEEE Standard for Verilog Hardware Description Language (IEEE Std 1364-2006), 2006.